

CREDIT RISK ASSESSMENT SYSTEM

Machine Learning Internship Project Report

Task Name	Credit Risk Assessment System
Intern Name	Seerat Un Nisa
Intern ID	CRH-2026-ML-025
Organization	Code Room Hub
Domain	Machine Learning
Dataset Name	Loan_Default.csv
Platform Used	Google Colab / Visual Studio Code
Evaluation Date	August 31, 2026

Prepared as a technical internship report documenting the complete machine-learning workflow, model evaluation, explainability, API development, containerization, version control, and cloud deployment of the project.

EXECUTIVE SUMMARY

This report documents the development of a Credit Risk Assessment System designed to estimate the probability that a loan applicant will default. The project follows an end-to-end machine-learning lifecycle: dataset inspection, exploratory data analysis, feature engineering, preprocessing, model training, model comparison, cross-validation, hyperparameter tuning, explainable AI, model persistence, REST API development, dashboard integration, Docker containerization, GitHub version control, and cloud deployment.

The working dataset, `Loan_Default.csv`, contains 255,347 records and 18 original columns. The target variable is `Default`, with a class distribution of approximately 88.39% non-default and 11.61% default observations. Because the positive class is substantially smaller, model evaluation was not based on accuracy alone; precision, recall, F1-score, ROC-AUC, and PR-AUC were considered.

Feature engineering expanded the data from 18 original columns to 24 columns. Engineered variables included `MonthlyIncome`, `EstimatedMonthlyDebt`, `LoanToIncome`, `CreditToAgeRatio`, `EmploymentStability`, `HighRiskCombo`, `InterestDebtStress`, and `LoanPerCreditLine`. After removing the identifier and target from the predictor matrix and applying preprocessing, the final production preprocessing output contained 39 model features.

Three primary classifiers were evaluated during the project: Logistic Regression, Random Forest, and HistGradientBoosting. The final production artifact used for the deployed API is a Logistic Regression model with 39 coefficients. The tuned Logistic Regression achieved approximately 0.8869 accuracy, 0.6150 precision, 0.0690 recall, 0.1240 F1-score, 0.7617 ROC-AUC, and 0.3366 PR-AUC on the held-out test set. The low recall indicates an important limitation: the deployed model is comparatively conservative at identifying default cases at the selected operating threshold.

The completed system exposes prediction functionality through FastAPI, provides a Streamlit dashboard for user interaction, packages the backend with Docker, stores model artifacts using joblib, and is deployed as separate production services. The backend was deployed on Railway and the dashboard was also deployed on Railway. The project source is maintained in a public GitHub repository under the Seerat-Un-Nisa account.

ACKNOWLEDGEMENT

I would like to express my appreciation to Code Room Hub for providing the internship environment in which this machine-learning project was developed. The internship provided practical exposure to data preprocessing, exploratory analysis, feature engineering, supervised classification, model evaluation, explainable AI, API development, Docker, Git/GitHub, and cloud deployment.

I also acknowledge the value of iterative testing and debugging throughout the project. Particular attention was given to preserving consistency between the training-time feature engineering and the production inference pipeline, as this is essential when a trained model is integrated into an external API.

The project was developed using Google Colab for the principal machine-learning workflow and Visual Studio Code for the application, deployment, and software-engineering stages.

DECLARATION

I declare that this report presents the work completed for the stated internship task and describes the implemented Credit Risk Assessment System. The technical results reported in this document are based on the project notebook, saved model artifacts, application implementation, testing outputs, and deployment configuration available for the project.

TABLE OF CONTENTS

1. Introduction
 2. Organization and Internship Context
 3. Problem Statement and Objectives
 4. Dataset Description
 5. Exploratory Data Analysis
 6. Feature Engineering
 7. Data Preprocessing
 8. Model Development and Comparison
 9. Cross-Validation and Hyperparameter Tuning
 10. Explainable AI with SHAP
 11. Model Persistence and Production Pipeline
 12. FastAPI REST API
 13. Streamlit Dashboard
 14. Docker Containerization
 15. GitHub Version Control
 16. Cloud Deployment and Testing
 17. Results and Discussion
 18. Limitations and Risk Considerations
 19. Skills and Learning Outcomes
 20. Conclusion and Future Work
- References
- Appendices

1. INTRODUCTION

Credit risk assessment is a classification problem in which borrower characteristics are used to estimate the likelihood of loan default. In a practical lending environment, such a system can support decisions by assigning a probability of default and translating that probability into an operational risk category.

The purpose of this internship project was to develop a complete, deployable machine-learning solution rather than a notebook-only experiment. The project therefore covers both the analytical and software-engineering sides of machine learning. The final system accepts borrower information, recreates the engineered features used during training, applies the saved preprocessing transformer, obtains a probability from the saved Logistic Regression model, and returns a risk assessment.

The implementation also demonstrates the importance of reproducibility. The trained estimator and preprocessing transformer were saved as joblib artifacts, the required scikit-learn version was pinned for compatibility, and the API was containerized using Docker. The production architecture separates the prediction service from the dashboard interface.

1.1 Project Scope

- Perform data inspection, cleaning, exploratory analysis, and feature engineering.
- Build a preprocessing pipeline for numerical and categorical variables.
- Compare multiple supervised classification algorithms.
- Evaluate performance using metrics appropriate for an imbalanced target.
- Apply cross-validation and hyperparameter optimization.
- Use SHAP-based explainability to inspect influential features.
- Export the selected model and preprocessing pipeline.
- Expose prediction functionality through a FastAPI REST service.
- Build and deploy a Streamlit dashboard.
- Containerize the API using Docker and maintain the project with Git/GitHub.

2. ORGANIZATION AND INTERNSHIP CONTEXT

Code Room Hub is the organization associated with this internship task. The internship domain for the project is Machine Learning. The work was structured around an applied machine-learning deliverable, requiring the intern to progress from raw data to an operational prediction service.

The practical environment combined Google Colab and Visual Studio Code. Google Colab was used for the principal notebook-based machine-learning workflow, while Visual Studio Code was used to organize the project repository, develop the API and dashboard, build Docker images, run local tests, manage Git commits, and prepare the deployment.

The internship therefore provided exposure to both data-science workflow and production-oriented engineering. This distinction is important because a model with good notebook results is not automatically a deployable application; the project required the trained artifacts, inference code, dependencies, API contract, user interface, and deployment environment to work together.

3. PROBLEM STATEMENT AND OBJECTIVES

The central problem is to estimate whether a loan applicant is likely to default using demographic, financial, employment, credit-history, and loan-related variables. The target variable Default is binary, with 0 representing

non-default and 1 representing default.

The project objectives were to construct a reproducible classification pipeline, identify useful risk-related patterns, compare alternative algorithms, tune the selected approach, explain model behavior, and deliver the model as an accessible application.

3.1 Specific Objectives

- Analyze the structure and quality of the Loan_Default.csv dataset.
- Quantify the target-class imbalance and identify relevant numerical and categorical variables.
- Create domain-inspired financial and risk indicators.
- Prevent preprocessing inconsistencies by fitting transformations on training data and reusing the fitted transformer.
- Compare Logistic Regression, Random Forest, and HistGradientBoosting.
- Use stratified cross-validation to assess model stability.
- Perform hyperparameter optimization for model improvement.
- Interpret model behavior using SHAP.
- Build a production inference API and interactive dashboard.
- Validate the deployed services with real HTTP requests.

4. DATASET DESCRIPTION

The project uses Loan_Default.csv. The original dataset contains 255,347 rows and 18 columns. The available variables describe borrower characteristics, employment information, loan characteristics, and the Default outcome.

Category	Variables
Identifiers / target	LoanID, Default
Numerical	Age, Income, LoanAmount, CreditScore, MonthsEmployed, NumCreditLines, InterestRate, LoanTerm, DTIRatio
Categorical	Education, EmploymentType, MaritalStatus, HasMortgage, HasDependents, LoanPurpose, HasCoSigner

4.1 Target Distribution

Class	Approx. proportion	Interpretation
0	88.39%	Non-default
1	11.61%	Default

The target distribution demonstrates class imbalance. Consequently, a high accuracy value by itself could be misleading because a classifier can obtain strong accuracy while identifying relatively few default cases. This is why ROC-AUC, PR-AUC, precision, recall, and F1-score were also tracked.

5. EXPLORATORY DATA ANALYSIS

Exploratory analysis focused on data types, unique categorical values, descriptive statistics, correlations, and the relationship between borrower variables and Default. The categorical fields were inspected for their possible values. Education contained four categories, EmploymentType four categories, MaritalStatus three categories, LoanPurpose five categories, and the Yes/No fields contained two categories each.

The numerical analysis identified several meaningful associations with default. In the original feature set, Age had a correlation of approximately -0.1678 with Default, InterestRate approximately 0.1313, Income approximately -0.0991, MonthsEmployed approximately -0.0974, LoanAmount approximately 0.0867, and CreditScore approximately -0.0342.

The correlation analysis was used as an exploratory diagnostic rather than as proof of causality. The relatively modest individual correlations also support the use of a multivariate model capable of combining information across several borrower attributes.

5.1 Engineered Feature Correlations

Engineered feature	Correlation with Default
LoanToIncome	0.1790
CreditToAgeRatio	0.1121
InterestDebtStress	0.1053
MonthlyIncome	-0.0991
EstimatedMonthlyDebt	-0.0553
LoanPerCreditLine	0.0550
HighRiskCombo	0.0203
EmploymentStability	0.0032

6. FEATURE ENGINEERING

Feature engineering was used to convert raw borrower variables into ratios and indicators that more directly represent financial burden, credit context, and employment stability. The engineered dataset increased from 18 original columns to 24 columns.

Feature	Purpose
MonthlyIncome	Approximate monthly income derived from annual Income.
EstimatedMonthlyDebt	Approximate monthly debt burden using MonthlyIncome and DTIRatio.
LoanToIncome	Loan amount relative to annual income.
CreditToAgeRatio	Credit score relative to age.
EmploymentStability	Months employed relative to age-based employment exposure.
HighRiskCombo	Binary flag for high DTI combined with low credit score.
InterestDebtStress	Interaction between InterestRate and DTIRatio.
LoanPerCreditLine	Loan amount relative to the number of credit lines.

A key production requirement was consistency: the same engineering formulas must be recreated when a new borrower is submitted to the API. The FastAPI implementation therefore contains an `engineer_features` function that reconstructs these variables before applying the saved preprocessor.

7. DATA PREPROCESSING

After feature engineering, LoanID was removed because it is an identifier rather than a predictive feature, and Default was separated as the target. The remaining variables were divided into numerical and categorical columns.

The data was split into training and test sets using an 80:20 stratified split. The training set contained 204,277 rows and the test set contained 51,070 rows. The preprocessing pipeline transformed the training data into 39 features and applied exactly the same fitted transformations to the test data.

Stage	Shape / Result
Original dataset	255,347 rows × 18 columns
After feature engineering	255,347 rows × 24 columns
Training set	204,277 rows
Testing set	51,070 rows
Processed training data	204,277 rows × 39 features
Processed testing data	51,070 rows × 39 features
Missing / invalid processed values	0 reported

8. MODEL DEVELOPMENT AND COMPARISON

The project evaluated multiple supervised classification models. The principal production comparison included Logistic Regression, Random Forest, and HistGradientBoosting. An experimental notebook phase also included XGBoost and Optuna-based optimization; this was treated as an experimental modeling stage rather than the final deployed artifact.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.8869	0.6168	0.0695	0.1249	0.7617	0.3367
Random Forest	0.8854	0.6471	—	—	0.7416	—
HistGradientBoosting	0.8865	0.6053	—	—	0.7572	—

The held-out test results show that Logistic Regression produced the strongest ROC-AUC among the compared production models. Its PR-AUC was also tracked because the default class is relatively uncommon.

The experimental notebook additionally recorded an XGBoost comparison with ROC-AUC 0.7428 and F1-score 0.3409, while the notebook's MLflow comparison phase recorded Logistic Regression, Random Forest, and XGBoost ROC-AUC values of 0.7616, 0.7490, and 0.7428 respectively. These values belong to the notebook's experimental modeling phase and should not be confused with the final production artifact.

9. CROSS-VALIDATION AND HYPERPARAMETER TUNING

Stratified cross-validation was used so that each fold retained a similar proportion of default and non-default cases. For the Logistic Regression evaluation, five-fold stratified cross-validation produced a mean ROC-AUC of approximately 0.7558 with a standard deviation of 0.0031. The corresponding mean PR-AUC was approximately 0.3264.

Validation method	Result
5-fold Stratified CV ROC-AUC mean	0.7558
5-fold Stratified CV ROC-AUC std	0.0031
5-fold Stratified CV PR-AUC mean	0.3264

Hyperparameter tuning was subsequently applied. The tuned Logistic Regression used $C \approx 0.074593$, `class_weight=None`, and `solver=liblinear`. Its best cross-validation ROC-AUC was approximately 0.7559. On the

held-out test set, the tuned model produced accuracy 0.8869, precision 0.6150, recall 0.0690, F1 0.1240, ROC-AUC 0.7617, and PR-AUC 0.3366.

The small difference between the baseline and tuned Logistic Regression results indicates that tuning improved the selected objective only marginally. This is a useful finding: optimization does not necessarily produce a large gain when the original model is already near its local optimum for the chosen feature representation and evaluation setup.

10. EXPLAINABLE AI WITH SHAP

Explainable AI was incorporated using SHAP (SHapley Additive exPlanations) to examine which features contributed most strongly to model outputs. The project inspected SHAP values over a sample of the processed test data and used individual borrower explanations to illustrate how a prediction can be interpreted.

Feature observed as influential	Interpretation in project context
Age	Borrower age contributed materially to model output.
InterestRate	Interest-rate level contributed to predicted risk.
LoanToIncome	Loan burden relative to income was an important engineered indicator.
MonthsEmployed	Employment duration contributed to risk estimation.
HasCoSigner	Co-signer status contributed through categorical encoding.
HasDependents	Dependent status also appeared among influential categorical features.

An individual borrower example in the project produced a default probability around 0.0302, with LoanToIncome, MonthsEmployed, EmploymentType_Full-time, Age, and HasCoSigner_Yes appearing among the top contributing factors. SHAP was used for interpretability rather than as a separate predictive model.

11. MODEL PERSISTENCE AND PRODUCTION PIPELINE

The production repository contains two important model artifacts: `logistic_regression_model.joblib` and `preprocessor.joblib`. The model contains one binary Logistic Regression classifier with a coefficient matrix of shape (1, 39), corresponding to 39 processed features. The model classes are [0, 1].

The production environment pins scikit-learn to version 1.6.1 and joblib to 1.5.3. This is important because scikit-learn's documentation warns that persisted models saved with one library version should not be assumed to be safely compatible with a different version. The deployment therefore uses the same scikit-learn version as the saved production artifacts.

The inference sequence is: receive JSON input → construct a DataFrame → recreate engineered features → apply saved preprocessing → compute default probability → derive risk score → assign risk category → assign decision → return JSON.

11.1 Production Risk Rules

Default probability	Risk category	Decision
< 0.15	Low Risk	APPROVED
0.15–0.35	Medium Risk	MANUAL REVIEW
> 0.35	High Risk	REJECTED

12. FASTAPI REST API

The FastAPI service provides a structured REST interface for the production model. The API defines a LoanApplication schema containing 16 borrower input fields. It loads the model and preprocessor at application startup and provides endpoints for basic service information and prediction.

Endpoint	Purpose	Observed status
/	Service information	200 OK
/health	Health status	200 OK
/model-info	Model and feature metadata	200 OK
/predict	Loan default risk prediction	200 OK
/docs	Interactive Swagger/OpenAPI interface	200 OK

A successful production-style prediction request returned a JSON response containing `default_probability`, `risk_score`, `risk_category`, and `decision`. For example, a low-risk sample produced a default probability of 0.0249, a risk score of 2.49, a Low Risk category, and an APPROVED decision.

The project initially encountered an API inference error when a request was sent against an incompatible local model environment. The saved model was subsequently loaded successfully after aligning scikit-learn to version 1.6.1. The deployed API was then tested successfully, including `/health`, `/model-info`, `/predict`, and `/docs`.

13. STREAMLIT DASHBOARD

A Streamlit dashboard was developed as the user-facing layer of the system. It collects borrower information through form controls and sends the data to the FastAPI prediction endpoint. The returned prediction is presented as a risk assessment result.

The dashboard displays the default probability, risk score, risk category, and loan decision. Screenshots were captured for the principal application states, including input form, approved result, manual-review result, rejected result, API pages, and the Streamlit interface.

A production integration issue occurred when the dashboard was configured to call `127.0.0.1:8000/predict`. That address refers to the dashboard container or local machine rather than the separately deployed backend. The dashboard was corrected to use the production FastAPI URL, after which the dashboard operated normally.

13.1 Risk Score Interpretation

In the final deployed API implementation, the risk score is explicitly calculated as $\text{default_probability} \times 100$ and rounded to two decimal places. Therefore a displayed probability of 19.84% and a risk score of 19.84 represent the same underlying quantity on different scales. This is internally consistent with the implemented code, although a future version could use a distinct credit-score-style scale if a separate risk-scoring methodology is required.

14. DOCKER CONTAINERIZATION

Docker was used to package the FastAPI backend together with its runtime dependencies and model artifacts. The Dockerfile uses `python:3.12-slim` as the base image, copies `api/requirements.txt`, installs the dependencies, copies the API source and model artifacts, exposes port 8000, and starts Uvicorn with the FastAPI application.

Docker component	Configuration
Base image	<code>python:3.12-slim</code>
Working directory	<code>/app</code>
Application server	Uvicorn

Docker component	Configuration
Container port	8000
Model artifacts	/app/models/
Entrypoint	uvicorn api.main:app --host 0.0.0.0 --port 8000

The Docker installation was verified with Docker version 29.7.2 and a successful hello-world container. The project image credit-risk-api was built successfully. Running the image exposed port 8000, and the container successfully served /docs and /predict. The container was inspected to verify that both model files were present.

The final Docker image was built for amd64, consistent with the available Docker Desktop environment used during testing.

15. GITHUB VERSION CONTROL

The project was initialized as a Git repository and committed as an initial project snapshot. The GitHub repository is hosted under the Seerat-Un-Nisa account:

<https://github.com/Seerat-Un-Nisa/Credit-Risk-Assessment-System>

The repository contains the API, dashboard, model artifacts, final notebook, screenshots, Docker configuration, dependency files, and documentation. A .gitignore was used to exclude the virtual environment, Python cache files, local environment files, macOS metadata, notebook checkpoints, logs, and temporary files.

The initial commit author information was corrected so that the Git commit is attributed to Seerat using the intended GitHub identity. The repository was then force-updated safely with --force-with-lease and verified as synchronized with origin/main.

16. CLOUD DEPLOYMENT AND TESTING

The backend and dashboard were deployed on Railway. Railway was selected after Render requested payment-card verification even for the intended free-tier workflow. The project was connected to the GitHub repository and deployed from the main branch.

Service	Production URL
FastAPI backend	https://credit-risk-assessment-system-production.up.railway.app
FastAPI Swagger	https://credit-risk-assessment-system-production.up.railway.app/docs
FastAPI root	https://credit-risk-assessment-system-production.up.railway.app/
FastAPI health	https://credit-risk-assessment-system-production.up.railway.app/health
FastAPI model info	https://credit-risk-assessment-system-production.up.railway.app/model-info
FastAPI prediction	https://credit-risk-assessment-system-production.up.railway.app/predict
Streamlit dashboard	https://dashboard-production-703e.up.railway.app/

Production verification confirmed that the API endpoints were responding correctly. The dashboard was also verified after updating its API target from the local address to the deployed backend URL. This resulted in a functioning end-to-end flow from browser input to cloud-hosted prediction service and back to the dashboard.

17. RESULTS AND DISCUSSION

The final system demonstrates a complete machine-learning application rather than an isolated model. The analytical component successfully processed more than a quarter-million records, engineered additional risk

indicators, transformed mixed numerical and categorical variables into 39 model features, trained and evaluated multiple classifiers, and performed cross-validation and tuning.

The Logistic Regression model achieved a ROC-AUC of approximately 0.7617 on the held-out test set. This indicates useful ranking ability between default and non-default cases, but it is not strong enough to treat the system as a fully autonomous lending decision maker. The precision of approximately 0.6150 means that, under the tested threshold, a substantial portion of positive predictions corresponded to actual defaults; however, recall of only approximately 0.0690 shows that most actual defaults were not captured at that threshold.

This precision-recall trade-off is the most important technical limitation of the final model. Accuracy of 0.8869 is close to the non-default majority proportion, so it should not be interpreted as evidence that 88.69% of default decisions are correct. PR-AUC of approximately 0.3366 provides additional information about performance under the imbalanced class distribution.

From an engineering perspective, the project successfully connected the trained artifact to a reproducible preprocessing pipeline and exposed it through a production API. This is a major improvement over a notebook-only workflow because the model can be consumed by another application through a documented HTTP interface.

18. LIMITATIONS AND RISK CONSIDERATIONS

- The production Logistic Regression model has low recall at the selected operating threshold, so it may miss many true default cases.
- The risk score in the deployed API is a direct percentage conversion of default probability rather than an independently calibrated credit score.
- The dataset represents historical observations; model behavior may change if the borrower population or lending environment changes.
- Correlation values describe association and should not be interpreted as causal effects.
- Joblib-based model persistence requires a trusted artifact and compatible package environment; scikit-learn explicitly warns about version compatibility.
- The current decision thresholds (<0.15 , $0.15\text{--}0.35$, >0.35) are engineering rules and should be validated against business costs, regulatory requirements, and acceptable risk levels before real lending use.
- The system should be treated as decision support rather than a substitute for human credit review until broader validation, calibration, fairness testing, and governance are completed.

A future production version should include probability calibration, threshold optimization based on explicit business costs, fairness/bias analysis across relevant groups, monitoring for data drift, model versioning, audit logging, and periodic retraining.

19. SKILLS AND LEARNING OUTCOMES

Area	Learning / Practical Outcome
Python & Pandas	Data loading, transformation, validation, and feature engineering.
EDA	Descriptive analysis, correlation analysis, class-distribution inspection, and visualization.
Machine Learning	Classification, train/test splitting, model comparison, and probability prediction.
Evaluation	Accuracy, precision, recall, F1-score, ROC-AUC, PR-AUC, and stratified cross-validation.
Optimization	Hyperparameter tuning and comparison of tuned versus baseline performance.
Explainability	SHAP-based global and individual feature contribution analysis.

Area	Learning / Practical Outcome
Software Engineering	FastAPI endpoint design, request validation, saved-model inference, and error debugging.
Frontend / Dashboard	Streamlit user interface and integration with a remote REST API.
DevOps	Docker image creation, container execution, port mapping, and environment consistency.
Version Control	Git repository management, commit identity, GitHub remote configuration, and deployment from main.
Cloud Deployment	Separate deployment of API and dashboard services using Railway.

20. CONCLUSION AND FUTURE WORK

The Credit Risk Assessment System achieved the principal objective of transforming a large loan-default dataset into a complete, deployable machine-learning application. The project progressed through the major stages of an applied ML lifecycle: EDA, feature engineering, preprocessing, model development, evaluation, cross-validation, tuning, explainability, model export, API development, dashboard development, Docker packaging, version control, and cloud deployment.

The final production architecture consists of a Logistic Regression model and saved preprocessing transformer behind a FastAPI service, with a Streamlit dashboard acting as the user interface. The system is accessible through deployed web services and can return a borrower risk assessment from structured input.

The strongest evidence of project completion is not only the reported model metrics but also the successful integration tests. The backend served Swagger documentation and prediction requests, the saved artifacts loaded correctly under scikit-learn 1.6.1, the Docker image executed successfully, and the deployed dashboard successfully communicated with the remote API.

Future work should focus on improving recall and calibration, evaluating alternative decision thresholds, performing formal fairness analysis, adding model monitoring, introducing automated CI/CD, improving the distinction between probability and risk score, and validating the system against a broader and more recent dataset. For production lending, human oversight and governance controls would remain essential.

REFERENCES

Scikit-learn documentation. Model persistence and compatibility guidance. https://scikit-learn.org/stable/model_persistence.html

FastAPI documentation. FastAPI framework and OpenAPI documentation.

Streamlit documentation. Streamlit application framework.

Docker documentation. Docker containerization and image concepts.

SHAP documentation. SHapley Additive exPlanations for model interpretability.

Optuna documentation. Hyperparameter optimization framework.

Project source repository: Seerat-Un-Nisa/Credit-Risk-Assessment-System on GitHub.

Project notebook: Credit_Risk_Assessment_System_FINAL.ipynb.

APPENDIX A — PROJECT STRUCTURE

.	Project root
api/main.py	FastAPI application and inference logic
api/requirements.txt	Backend dependencies
dashboard/app.py	Streamlit dashboard
models/logistic_regression_model.joblib	Saved production Logistic Regression model

.	Project root
models/preprocessor.joblib	Saved production preprocessing transformer
notebook/Credit_Risk_Assessment_System_FINAL.ipynb	Final machine-learning notebook
Dockerfile	Backend container definition
.dockerignore	Docker build exclusions
.gitignore	Git exclusions
requirements.txt	Environment dependency snapshot
screenshots/	Testing and UI screenshots

APPENDIX B — SAMPLE PRODUCTION PREDICTION

Input	Example value
Age	35
Income	60,000
LoanAmount	15,000
CreditScore	700
MonthsEmployed	60
NumCreditLines	4
InterestRate	8.5
LoanTerm	36
DTIRatio	0.30
Education	Bachelor's
EmploymentType	Full-time
MaritalStatus	Married
HasMortgage	Yes
HasDependents	Yes
LoanPurpose	Home
HasCoSigner	Yes

Observed API response for this sample: default_probability = 0.0249; risk_score = 2.49; risk_category = Low Risk; decision = APPROVED.

APPENDIX C — DEPLOYMENT CHECKLIST

Item	Status
Final notebook completed and saved	Completed
Model and preprocessor exported	Completed
Production scikit-learn version aligned	Completed
FastAPI endpoints tested	Completed

Item	Status
Swagger documentation tested	Completed
Streamlit dashboard tested	Completed
Docker image built and run	Completed
GitHub repository created	Completed
README completed	Completed
Backend deployed	Completed
Dashboard deployed	Completed